Project AgentTwin: Graduation Capstone Proposal

Execution-Grounded Differential Testing for Autonomous AI Agent Systems

Duration: 12 Weeks | **Team:** 5 Engineers | **Stack:** Python, LangGraph, OpenTelemetry

1. PROBLEM STATEMENT & OBJECTIVE

Autonomous agents can return valid text outputs while failing internally. Updates to prompts, models, or tool schemas frequently lead to omitted authorization checks, privilege escalation, or corrupted state. Output-based benchmarks and single-prompt LLM judges evaluate only the final string, missing intermediate policy violations and control-flow regressions. **Objective & Scope:** Build and evaluate AgentTwin on a LangGraph customer operations agent with tool interfaces (`query_db`, `modify_user`, `issue_refund`, `notify_customer`). AgentTwin runs a baseline and candidate agent on identical tasks in matched sandboxes, captures OpenTelemetry execution traces, and identifies the earliest divergent step causing behavioral or security regressions.

- **Hypothesis:** Paired trace-diff testing detects tool-call and policy regressions with higher accuracy than output-only evaluation and single-prompt LLM judges.
- **Classification Output:** Every run is classified as: **(1) Intended Evolution**, **(2) Unintended Regression**, **(3) Policy Violation**, or **(4) Inconclusive/Flaky**.

2. COMPARISON WITH EXISTING TOOLING

- **Agent Observability (LangSmith, Braintrust):** Logs execution traces and runs evaluators on static test sets; lacks paired execution across dual active versions.
- **Code Differential Testing (DiffTestGen, Testora):** Generates tests across code commits using call graphs; does not handle tool permissions, memory state, or stochastic LLM sampling.
- **Telemetry Standard:** Uses OpenTelemetry GenAI Semantic Conventions (`agent.run`, `tool.call`, `model.inference`) instead of a custom schema.

3. SYSTEM ARCHITECTURE & SCOPE REDUCTION

Component	Concept Paper Scope	12-Week Feasible Scope
Agent Framework	Arbitrary frameworks via custom adapters	Single LangGraph tool-calling agent
Tool Sandbox	Multi-container Docker isolation	In-process mocked tools with permission decorators (`read`, `write`, `net`, `pay`)
Trace Schema	Custom event specification	OpenTelemetry GenAI Semantic Conventions extended for memory/policy events
Change Intake	Unstructured PR and commit parsing	Structured YAML intent spec defining invariants and forbidden behavior
Evaluators	4 full LLM evaluator agents	Deterministic assertions + 2 evaluators (Functional, Policy) + LLM fallback
Adjudication	Multi-agent LLM consensus board	Rule-based trace alignment (earliest divergence) + $N = 3$ trial verification
Baselines	5 evaluation suites	3 baselines: (1) Output-only judge, (2) Multi-judge, (3) Trace-diff ablation

4. RESPONSIBILITIES, MILESTONES & DELIVERABLES

- **Team Roles: Person A (Harness Lead):** LangGraph agent, mocked tool decorators, OpenTelemetry instrumentation. **Person B (Benchmark Lead):** Curate 40–60 seeded mutation scenarios across 4 classes (prompt edits, tool schemas, memory, model swaps). **Person C (Evaluator Lead):** Assertion DSL, Functional evaluator, Policy evaluator. **Person D (Diff Engine Lead):** Sequence alignment, earliest-divergent-step detector, $N = 3$ verification harness. **Person E (Integration Lead):** Pipeline orchestration, 3-baseline comparative experiments, metrics collection, web dashboard.
- **Weeks 1–4 (Harness):** Intent YAML schema, OpenTelemetry trace mappings, agent runner, and mock-tool sandboxes.
- **Weeks 5–8 (Engine & Tests):** 40–60 scenario benchmark generation, assertion DSL, sequence alignment algorithm, $N = 3$ reproduction loop.
- **Weeks 9–12 (Evaluation):** Execute 3 baselines, collect metrics, build inspection UI, complete final report.
- **Software Deliverables:** (1) `agenttwin-core` Python CLI/engine; (2) 50-scenario benchmark suite with ground-truth mutation manifests; (3) Web trace-diff dashboard; (4) Capstone final report.
- **Target Metrics:** $\geq 85\%$ detection recall on seeded regressions; 100% recall on unauthorized tool calls; $\geq 80\%$ accuracy on earliest divergent step localization; $\leq 10\%$ false-positive rate on benign changes.

5. TECHNICAL RISKS & MITIGATIONS

- **Token Cost:** Paired runs increase model calls. *Mitigation:* Mock tools run locally in Python with zero API token cost; Tier 1 assertions and Tier 2 trace diffs use zero LLM tokens on clean passes and hard policy breaks.
- **Sampling Non-Determinism:** Random output variations cause false-positive diffs. *Mitigation:* Regressions require confirmed divergence across $N = 3$ repeated trials.
- **Harness Bottleneck (Weeks 3–4):** Downstream work depends on trace format. *Mitigation:* Person A generates static OpenTelemetry trace fixtures in Week 2 so Persons B and C can develop in parallel.